

React moderno na prática

Módulo: 09

Aula: 06 — Context, Provider e Yarn no projeto

Instrutor: Lucca Dumas

CONTEÚDO

1) Um pulinho na aula anterior

Na **Aula 05**, vimos **Hooks** (`useState` , `useEffect` , `useContext`), uma **introdução** a **Context + Provider**, **ReactDOM x DOM**, `export function` e **Yarn**.

Nesta aula, aprofundamos **como o frontend compartilha estado: Context + Provider** (como nos toasts) e **Zustand** (como na **empresa ativa** após o login), além do **Yarn** no repositório.

Nos trechos de Context desta apostila usamos `useState` no Provider. `useCallback` e `useMemo` ficam para outro momento do curso.

2) O que veremos hoje?

1. **Empresa ativa depois do login** — problema, **Zustand** no ERP e **laboratório com Zustand** no projeto de treino
2. **Context API** — recapitulação: `createContext` , `Provider` , consumo com `useContext` / hook
3. **Duas abordagens** — quando o projeto usa **Context** e quando usa **store (Zustand)**
4. **Toasts** — `ToastProvider` , `CustomToastProvider` , `useToast` / `useCustomToast`
5. **Yarn** — instalar, scripts, `yarn.lock`
6. **Hands-on, erros comuns e exercícios**

3) Empresa depois do login: por que compartilhar estado?

Fluxo no produto:

1. O usuário **faz login**.
2. Abre a tela de **seleção de empresa** (matriz/filial).

3. Depois de confirmar, o sistema precisa saber, em **qualquer tela**, qual empresa está ativa — **cabeçalho, filtros, empresaId** em requisições, etc.

Se tudo descesse **somente por props** (App → Layout → Header → ...), voltamos ao **prop drilling**.

No **repositório real**, esse estado fica na **store Zustand** `src/store/empresaStore.ts` , consumida com **useEmpresaStore()** (`src/store/useEmpresaStore.ts`). A tela `src/screens/Login/SelecaoEmpresa/index.tsx` chama `setEmpresaId` / `setEmpresaDados` ao confirmar; O `src/components/navigation/Header.tsx` lê `empresaId` e `empresaDados` .

Ou seja: **estado global fora da árvore de Providers do React**, acessível de qualquer componente que importe o hook da store.

4) Context API (recapitulação)

Peça	Papel
<code>createContext</code>	Cria o canal pelo qual o valor será lido.
<code><MeuContext.Provider value={...}></code>	Disponibiliza o valor para todos os filhos na árvore.
<code>useContext(MeuContext)</code>	Lê o valor (em geral dentro de um hook <code>useMeuContexto()</code>).

O valor costuma incluir **estado** (`useState` no Provider) e **funções** que alteram esse estado.

5) Duas abordagens no frontend: Context x Zustand

O mesmo repositório usa as duas, para **problemas diferentes**:

	Context + Provider	Zustand (store)
Onde "mora" o estado	No React, dentro do Provider na árvore	Na store (<code>createStore</code>), ligada ao React via useStore
Quem enxerga	Componentes descendentes do Provider	Qualquer componente que chame o hook da store (sem envolver a árvore com Provider)
Exemplo no projeto	Toasts: <code>ToastContext</code> , <code>CustomToastContext</code> , Providers no <code>App.tsx</code>	Empresa ativa: <code>empresaStore.ts</code> , <code>useEmpresaStore.ts</code>

	Context + Provider	Zustand (store)
Quando costuma aparecer na discussão	Estado de UI acoplado à árvore (tema, toast, wizard interno)	Estado global de aplicação usado em muitas rotas (empresa, flags de sistema)

Não é regra universal de mercado: outros times podem colocar empresa em Context ou toasts em outra lib. Aqui o importante é **ler o código do projeto** e reconhecer os dois padrões.

6) Toasts no mesmo projeto (Context na prática)

Para **avisos** (sucesso, erro...), o frontend usa Context:

- `src/contexts/ToastContext.tsx` — `showToast`, `hideToast`, hook `useToast`.
- `src/contexts/CustomToastContext.tsx` — `showCustomToast`, `hideCustomToast`, hook `useCustomToast`.

No `src/App.tsx`:

```
function App() {
  return (
    <ToastProvider>
      <CustomToastProvider>
        <AppContent />
      </CustomToastProvider>
    </ToastProvider>
  );
}
```

Componentes precisam estar **abaixo** desses Providers para usar `useToast()` ou `useCustomToast()`.

Consumo típico (trecho de referência):

```
import { useToast } from '../contexts/ToastContext';

const { showToast } = useToast();
// showToast({ message: '...', type: 'success', ... })
```

Abra `src/contexts/`: o desenho é **estado no Provider** + **funções no value**.

Nos Providers de toast do código de produção há recursos adicionais voltados a performance; o tema entra em aulas posteriores.

7) Laboratório no treino: empresa com Zustand (react-ts-vite-app)

Objetivo: após “escolher” a empresa, **ver o mesmo estado** no cabeçalho e na área de seleção, sem passar props entre eles — **alinhado ao ERP**, usando Zustand.

No template `react-ts-vite-app` (após `npm install zustand`), a estrutura é:

Arquivo	Papel
<code>src/store/empresaTreinoStore.ts</code>	<code>createStore</code> (<code>zustand/vanilla</code>): estado <code>empresaId</code> , <code>nomeEmpresa</code> e ações <code>definirEmpresa</code> , <code>limparEmpresa</code> .
<code>src/store/useEmpresaTreinoStore.ts</code>	Hook <code>useEmpresaTreinoStore</code> : <code>useStore(empresaTreinoStore)</code> para componentes React assinarem atualizações.
<code>src/components/CabecalhoEmpresa.tsx</code>	Lê <code>empresaId</code> e <code>nomeEmpresa</code> pela store.
<code>src/components/SelecaoEmpresa.tsx</code>	Chama <code>definirEmpresa</code> / <code>limparEmpresa</code> na mesma store.
<code>src/App.tsx</code>	Não precisa de <code>EmpresaAtivaProvider</code> : renderiza só os componentes.

Trecho — store (`empresaTreinoStore.ts`):

```
import { createStore } from "zustand/vanilla";

export type EmpresaTreinoState = {
  empresaId: number | null;
  nomeEmpresa: string;
  definirEmpresa: (id: number, nome: string) => void;
  limparEmpresa: () => void;
};

export const empresaTreinoStore = createStore<EmpresaTreinoState>((set) => ({
  empresaId: null,
  nomeEmpresa: "",
  definirEmpresa: (id, nome) => set({ empresaId: id, nomeEmpresa: nome }),
  limparEmpresa: () => set({ empresaId: null, nomeEmpresa: "" }),
})));
```

Trecho — hook (`useEmpresaTreinoStore.ts`):

```
import { useStore } from "zustand";
import { empresaTreinoStore } from "../empresaTreinoStore";
```

```
export function useEmpresaTreinoStore() {  
  return useStore(empresaTreinoStore);  
}
```

Consumo nos componentes:

```
const { empresaId, nomeEmpresa } = useEmpresaTreinoStore();  
// ou  
const { definirEmpresa, limparEmpresa } = useEmpresaTreinoStore();
```

No ERP, `empresaStore` também persiste em `localStorage`; no treino a store é só em memória, para focar no fluxo React + Zustand.

Passos sugeridos em aula

1. Conferir dependência `zustand` no `package.json` do treino.
2. Rodar o servidor de desenvolvimento e testar **Alpha**, **Beta** e **Limpar**.
3. Observar o **cabeçalho** atualizar em sincronia com os botões.
4. Comparar com o `empresaStore.ts` do frontend: mesma ideia de `createStore` + hook com `useStore`.

8) Boas práticas

- **Context**: um contexto por **assunto** (notificações, tema...), não um "mega contexto".
- **Context**: hook com **erro claro** se faltar o Provider.
- **Zustand**: manter **ações** (`definirEmpresa`, etc.) na store, evitando espalhar `set` manual em muitos arquivos.
- Quando **um único** componente usa o dado, `useState` **local** costuma ser suficiente.

9) Yarn no fluxo do projeto

O Yarn lê o `package.json`, instala dependências e executa **scripts** definidos no repositório **frontend**.

Comandos essenciais

- Instalar dependências:

```
yarn install
```

- Rodar ambiente de desenvolvimento:

```
yarn dev
```

(Em alguns projetos também existe `yarn start` apontando para o mesmo fluxo do Vite.)

- Gerar build:

```
yarn build
```

- Rodar preview do build:

```
yarn preview
```

- Lint e testes (conforme `package.json`):

```
yarn lint  
yarn test
```

Sobre o `yarn.lock`

- Garante versões reproduzíveis entre máquinas e CI.
- Deve ser **versionado** no Git.
- **Não** deve ser editado manualmente.

Adicionar pacote (com alinhamento do time)

```
yarn add nome-do-pacote  
yarn add -D nome-do-pacote
```

(`-D` = dependência de desenvolvimento.)

10) Hands-on da aula

Objetivo do hands-on

- No **frontend**, localizar **Providers** de toast no `App.tsx` e a store `empresaStore.ts` usada com `useEmpresaStore`.

- No **treino** `react-ts-vite-app`, rodar `npm run dev` (ou o gerenciador adotado na turma), testar **seleção de empresa** com Zustand e observar o **cabeçalho**.

Passos

1. Abrir `src/App.tsx`, `src/contexts/ToastContext.tsx` e `src/store/empresaStore.ts` no repositório **frontend**.
2. Abrir `src/screens/Login/SelecaoEmpresa/index.tsx` e notar chamadas à store.
3. No **treino**, abrir `src/store/empresaTreinoStore.ts` e os dois componentes de empresa.
4. Executar o servidor de desenvolvimento e validar os três botões.

11) Erros comuns desta aula

1. Usar `useToast` fora do `ToastProvider`
Resultado: erro em runtime (mensagem explícita no código).
2. Procurar um `EmpresaProvider` no ERP para a empresa ativa
No projeto real a empresa está na **store Zustand**, não em um Provider de Context com esse nome.
3. Colocar Provider só em volta de parte da árvore e consumir Context fora dele
Resultado: valor `undefined` ou erro no hook customizado.
4. Alterar `yarn.lock` na mão
Resultado: inconsistência de dependências entre ambientes.
5. Importar a store mas esquecer de usar `useStore` em componente React
Fora de componentes, ainda é possível usar `empresaTreinoStore.getState()` (avançado); em telas, prefira o **hook** para re-renderizar ao mudar o estado.

12) Exercícios de fixação

1. Context serve principalmente para:
 - () Substituir todo `useState` do projeto
 - (X) Compartilhar dados entre componentes sem passar props em cada nível
 - () Compilar TypeScript
 - () Gerar o `yarn.lock`
2. No ERP, a empresa escolhida após o login fica principalmente em:
 - () Só no componente da tela de seleção
 - (X) Uma store global (`empresaStore` / `useEmpresaStore`, Zustand) acessível em várias telas

- ☐ No `yarn.lock`
 - ☐ Apenas no servidor
3. Usar `useToast` fora do `ToastProvider` :
- ☐ Funciona com valor padrão
 - ☒ Gera erro explícito no desenvolvimento (no nosso código)
 - ☐ Desliga o Vite
 - ☐ Só em produção
4. O `yarn.lock` deve ser:
- ☐ Ignorado no Git
 - ☒ Versionado no Git e não editado manualmente
 - ☐ Apagado antes de cada deploy
 - ☐ Substituído pelo `package.json`
5. O `value` do `Provider` de `Context` fica disponível para:
- ☐ Só o filho direto
 - ☒ Todos os descendentes na árvore sob esse `Provider`
 - ☐ Só arquivos em `src/contexts/`
 - ☐ O Node no servidor
6. No projeto de treino, a troca de empresa simulada usa:
- ☒ Zustand (`empresaTreinoStore` + `useEmpresaTreinoStore`)
 - ☐ Redux Toolkit
 - ☐ Apenas `sessionStorage` sem React
 - ☐ Um `EmpresaProvider` obrigatório em volta do `App`

13) Desafio para casa

1. Abrir `src/store/empresaStore.ts` no **frontend** e listar **quais campos** existem em `EmpresaDados` (ou no estado da store) e o que `setEmpresaId` altera.
2. Abrir `ToastContext.tsx` e localizar o `useState` : **o que** ele representa em relação ao toast?
3. Na pasta do **frontend**: `yarn install` e `yarn dev` — uma linha para cada comando, dizendo o que observou.
4. No **treino**, desenhar um esquema: `empresaTreinoStore` no centro, setas a partir de `SelecaoEmpresa` e `CabecalhoEmpresa` .

Regras

- Caminhos `src/...` referem-se ao **frontend** do ERP, salvo menção ao `react-ts-vite-app` .
- No treino, manter o padrão **store em `.ts` + hook `useStore`** em arquivo dedicado.

14) Resumo final

Ao final desta aula, o aluno deve conseguir:

- explicar **estado global** para **empresa ativa** e apontar `empresaStore` / `useEmpresaStore` , `SelecaoEmpresa` e `Header` no ERP;
- contrastar **Context + Provider** (toasts) com **Zustand** (empresa e outras stores em `src/store/`);
- localizar **toasts** no `App.tsx` e em `src/contexts/` ;
- reproduzir o **laboratório** no `react-ts-vite-app` com `empresaTreinoStore` e `useEmpresaTreinoStore` ;
- usar **Yarn** (`install` , `dev` , `build` , `preview` , `lint` , `test`) e o papel do `yarn.lock` .

Na continuação do curso, aprofundaremos `useMemo` / `useCallback` e outros detalhes de produção, quando fizer parte do programa.